



Politecnico di Bari

Corsi:

Distributed measurement and data acquisition systems
&
Advanced data acquisition and intelligent data processing

Tema d'anno: Progetto LiDAR

Docente: Attilio di Nisio

Autori:

Francesco Faienza

Matricola: 596856

Email: f.faienza@studenti.poliba.it

Walter Amicarelli

Matricola: 598436

Email: w.amicarelli@studenti.poliba.it

Pantaleo Grosso

Matricola: 598324

Email: p.grosso1@studenti.poliba.it

Indice

1	Introduzione	1
1.1	Obiettivi del progetto	1
1.2	LiDAR: descrizione e funzionamento	1
1.2.1	Triangolazione	3
1.3	Origini e sviluppo del LiDAR	3
1.4	Evoluzione e trend futuri	3
2	LiDAR e configurazione	5
2.1	Strumenti utilizzati	5
2.1.1	Kit Arduino	5
2.1.2	ESP32-WROOM-32	6
2.1.3	4-Bit Bidirectional Voltage Level Converter	6
2.1.4	Sensore	6
2.2	Configurazione	8
2.2.1	Schematica	8
2.3	Protocollo di comunicazione	8
2.3.1	Esempio frame	11
2.4	Voltage Level Converter	12
3	Acquisizione Dati	13
3.1	Funzione GetData()	13
3.2	Estrazione Dati	14
3.3	Output del microcontrollore	15
4	Visualizzazione Dati	17
4.1	Diagramma polare	18
4.1.1	Rappresentazione polare	18
4.2	Estrazione dati Python	18
5	Debugging	20
5.1	Problematiche riscontrate	20
5.2	Analisi e risoluzione delle problematiche	20
5.2.1	Incompatibilità con Arduino Uno R3	20
5.2.2	Disallineamento dei livelli di tensione	21
5.2.3	Gestione della comunicazione e raccolta dati tramite porta seriale	21
6	Risultati e conclusioni	22

1 Introduzione

1.1 Obiettivi del progetto

L'obiettivo principale del progetto è stato quello di studiare il funzionamento del LiDAR, analizzando i protocolli di comunicazione e le modalità di acquisizione dei dati. In particolare, ci siamo concentrati sull'integrazione del sensore con un sistema basato su microcontrollore, sviluppando un codice in ambiente Arduino per ottimizzarne l'efficienza operativa.

Per garantire il corretto funzionamento del sensore LiDAR con il microcontrollore, abbiamo utilizzato un Voltage Level Converter, necessario per adattare i livelli di tensione tra i dispositivi. Inoltre, l'integrazione con un ESP32-WROOM-32 ha permesso di gestire la comunicazione e l'elaborazione dei dati in modo efficiente, sfruttando la potenza di calcolo del modulo.

I passaggi chiave del progetto sono due: comprendere il funzionamento del protocollo di comunicazione del sensore LiDAR e utilizzarlo con un hardware meno performante rispetto alla soluzione fornita dall'azienda produttrice.

L'intero progetto è stato sviluppato con l'obiettivo di massimizzare la precisione e la stabilità delle misurazioni, assicurando che il LiDAR fosse perfettamente operativo nelle condizioni di utilizzo previste.

1.2 LiDAR: descrizione e funzionamento

Il **LiDAR** (Light Detection and Ranging) è una tecnologia di telerilevamento che utilizza impulsi laser per misurare con elevata precisione la distanza da un oggetto o da una superficie. Questa tecnica si basa sul principio della triangolazione, secondo cui un impulso laser viene emesso, riflesso dall'oggetto target e successivamente rilevato da un sensore. Misurando l'angolo di ritorno del raggio rispetto alla posizione del trasmettitore e del ricevitore, è possibile determinare la posizione esatta dell'oggetto in un sistema bidimensionale. Il LiDAR si compone principalmente di tre elementi fondamentali:

- una sorgente laser;
- un sistema di ricezione;
- un'unità di elaborazione.

La sorgente laser emette impulsi luminosi ad alta frequenza, che possono essere nell'infrarosso, nel visibile o nell'ultravioletto, a seconda dell'applicazione specifica.

Le applicazioni del LiDAR sono molteplici e spaziano in diversi settori. In ambito geospaziale, questa tecnologia è ampiamente utilizzata per la creazione di modelli digitali del terreno (DTM) e di modelli digitali di elevazione (DEM), consentendo una rappresentazione accurata della morfologia terrestre. Nel settore automobilistico, il LiDAR è una componente chiave dei sistemi di guida autonoma, in quanto permette ai veicoli di rilevare ostacoli e mappare l'ambiente circostante con un alto grado di precisione. Altre applicazioni includono il monitoraggio forestale, la rilevazione archeologica, la navigazione aerea e marittima, nonché il supporto alle operazioni di soccorso e protezione civile in caso di calamità naturali.

Una delle principali caratteristiche che rende il LiDAR particolarmente vantaggioso rispetto ad altre tecnologie di telerilevamento, come il radar o la fotogrammetria, è la sua elevata risoluzione spaziale. Grazie alla possibilità di emettere milioni di impulsi al secondo, il LiDAR è in grado di generare mappe ad alta densità, offrendo una rappresentazione estremamente dettagliata dell'ambiente. Tuttavia, presenta anche alcune limitazioni, come la sensibilità alle condizioni atmosferiche (nebbia, pioggia e polvere possono interferire con la precisione delle misurazioni) e il costo relativamente elevato delle apparecchiature e delle operazioni di rilevamento.

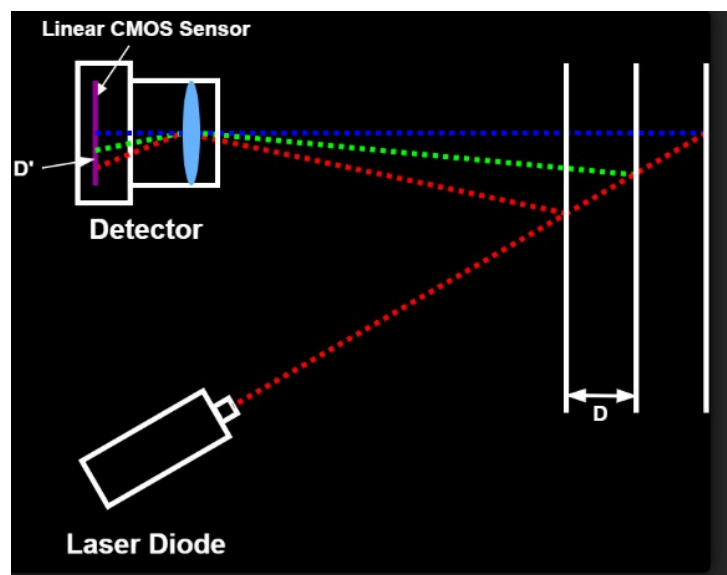


Figura 1.1: Triangolazione LiDAR

1.2.1 Triangolazione

Il LiDAR emette un fascio di luce con una determinata angolazione. Quando il fascio colpisce una superficie, la luce riflessa non segue una traiettoria rettilinea, ma subisce una deviazione di un piccolo angolo. L'entità di questa deviazione dipende dalla distanza della superficie riflettente, stabilendo così una relazione diretta tra l'angolo di deviazione e la distanza stessa.

Per determinare lo spostamento del punto laser e calcolare la distanza dalla superficie, il sistema si avvale di un sensore di immagini e di una lente ottica. Quest'ultima raccoglie la luce riflessa e la mette a fuoco sul sensore, consentendo di individuare con precisione la posizione del punto riflesso e, di conseguenza, di stimare la distanza dell'oggetto rilevato come illustrato nella figura 1.1.

1.3 Origini e sviluppo del LiDAR

Il LiDAR ha origine negli anni '60, inizialmente sviluppato per applicazioni aerospaziali e militari. Con il tempo, la tecnologia si è evoluta e diffusa in molti altri settori, grazie ai progressi nei laser, nei sensori e nei sistemi di elaborazione dati. Negli anni '80 e '90, il LiDAR ha trovato impiego nella cartografia e nella topografia, mentre oggi viene integrato in dispositivi portatili, droni e veicoli autonomi.

1.4 Evoluzione e trend futuri

Negli ultimi anni, la tecnologia LiDAR ha subito un'evoluzione significativa, con lo sviluppo di sensori sempre più compatti, efficienti e a basso costo. Una delle innovazioni più promettenti è il LiDAR a stato solido, che elimina le parti meccaniche mobili, aumentando l'affidabilità e riducendo i costi di produzione. Inoltre, l'integrazione con l'intelligenza artificiale e il cloud computing sta permettendo una gestione più avanzata dei dati raccolti, migliorando la precisione e la rapidità di elaborazione.

Conclusioni

In sintesi, il LiDAR rappresenta una tecnologia di telerilevamento avanzata e versatile, capace di fornire dati estremamente accurati in numerosi campi applicativi. La continua evoluzione dei sensori e delle tecniche di elaborazione sta ampliando ulteriormente il suo utilizzo, rendendolo una risorsa sempre più imprescindibile per la ricerca scientifica, l'industria e l'innovazione tecnologica.

2 LiDAR e configurazione

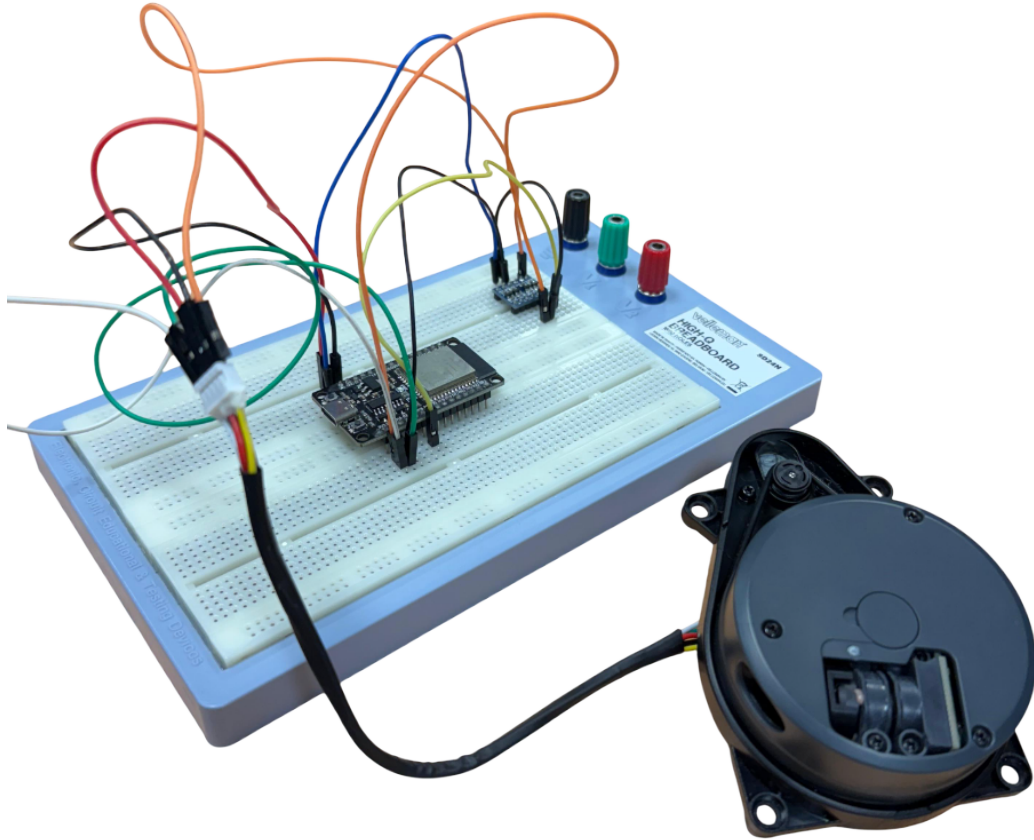


Figura 2.1: Setup Arduino e LiDAR collegato

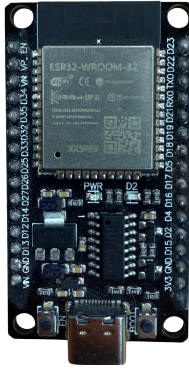
2.1 Strumenti utilizzati

2.1.1 Kit Arduino

- High-Q Breadboard;
- cablaggio;

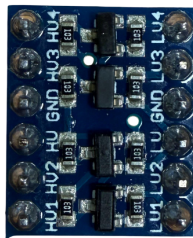
2.1.2 ESP32-WROOM-32

- modulo ESP32;



- cavo USB Type-A to USB Type-C;

2.1.3 4-Bit Bidirectional Voltage Level Converter



2.1.4 Sensore

- Delta-2A LiDAR;

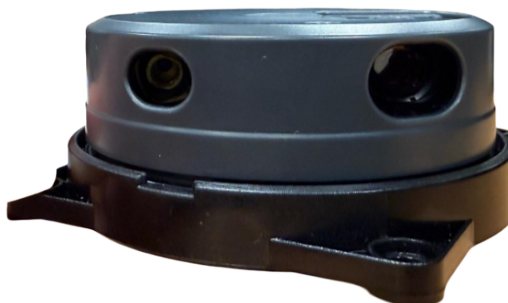


Figura 2.2: sensore LiDAR

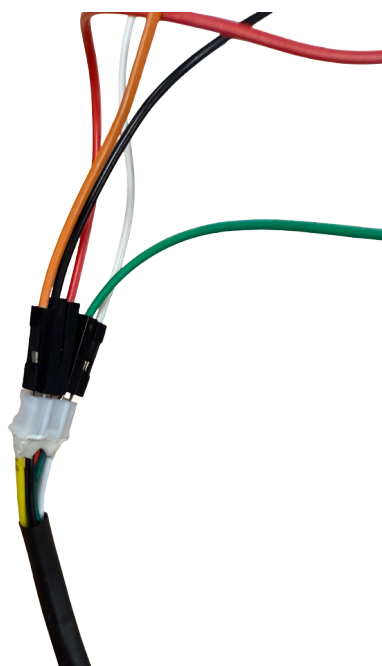


Figura 2.3: LiDAR senza copertura superiore



Figura 2.4: LiDAR senza copertura inferiore

- connettore a 5 PIN;



2.2 Configurazione

2.2.1 Schematica

In figura 2.5 è rappresentata la schematica del circuito utilizzato per l'acquisizione dati. Si sottolinea che l'ESP32 eroga 5V dal pin VIN poiché collegato tramite USB-C alla macchina utilizzata per la visualizzazione.

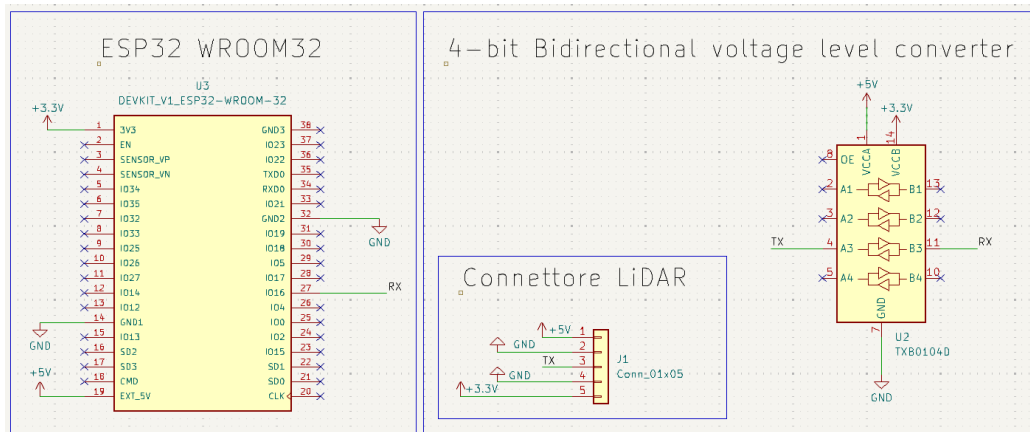


Figura 2.5: Schematica creata su KiCad

2.3 Protocollo di comunicazione

Il sensore impiegato nel progetto è il LiDAR Delta 2A, distribuito da 3irobotix. L'analisi della documentazione tecnica fornita dal produttore ha consentito di ricostruire la struttura del protocollo di comunicazione utilizzato dal dispositivo.

Tutti i dati trasmessi dal sensore sono rappresentati in formato esadecimale. Durante il funzionamento, il LiDAR esegue una scansione rotativa dell'ambiente circostante, acquisendo informazioni relative alla distanza e all'angolo di ciascun punto rilevato. L'intera circonferenza di 360° viene suddivisa in 16 frame di ampiezza 22.5° ciascuno.

All'interno di ogni frame, il sensore effettua un numero variabile di misurazioni. Il numero totale di punti acquisiti in un singolo frame si determina effettuando un conteggio dei dati sulle distanze presenti nel pacchetto informativo trasmesso dal sensore.

Il messaggio decodificato relativo al frame è strutturato come nella tabella 2.1.

Byte:	Significato:
0	intestazione
1 e 2	lunghezza frame
3	protocollo
4	tipo frame
5	command word
6 e 7	lunghezza payload
8	velocità
9 e 10	angolo offset
11 e 12	angolo start
(13+3N)	separatore
(14+3N)	distanza (MSB)
(15+3N)	distanza (LSB)
N-2	CRC
N-1	CRC

Tabella 2.1: Struttura frame

I byte evidenziati in giallo assumono valori costanti

- **byte 0: intestazione.**
È fisso a 0xAA e rappresenta l'inizio del messaggio relativo al frame;
- **byte 1 e 2: lunghezza frame.**
Indicano la lunghezza del messaggio dall'intestazione fino al byte precedente al CRC;
- **byte 3: protocollo.**
È fisso a 0x00 se il messaggio ha la command word AE, oppure fisso a 0x01 se la command word è AD. Indica il protocollo del messaggio;
- **byte 4: tipo frame.**
È fisso a 0x61 e indica il tipo del frame;

- **byte 5: command word.**

Può assumere due valori:

- 0xAE, corrispondente a un messaggio di tipo "Device Health", che comprende solo il valore della velocità;
- 0xAD, corrispondente a un messaggio di tipo "Measurement Information", che indica una scansione del sensore.

Il primo tipo di messaggio viene inviato in caso di malfunzionamento del laser.

- **byte 6 e 7: lunghezza payload.**

Indica la lunghezza del payload, contenente i dati delle misurazioni relative al frame;

- **byte 8: velocità.**

Valore riferito alla velocità di rotazione del sensore.

Se moltiplicato per 0.05, si ottiene il numero di rotazioni al secondo (RPS); se moltiplicato per 3, si ottiene il numero di rotazioni al minuto (RPM);

- **byte 9 e 10: angolo di offset.**

Valore a 16 bit con segno che indica l'angolo da aggiungere o sottrarre per allineare l'angolo di partenza della scansione alla realtà fisica;

- **byte 11 e 12: angolo di start.**

Valore a 16 bit senza segno che indica l'angolo di partenza della scansione relativa al frame. Per ottenere il valore effettivo è necessario convertirlo da esadecimale in decimale e moltiplicarlo per 0.01°. Per calcolare l'angolo di ogni misurazione all'interno di un frame, è necessario conoscere:

1. **numero di misurazioni totali**, calcolabile con la formula:

$$M = \frac{(\text{Lunghezza Payload} - 5)}{3} \quad (2.1)$$

dove M è il numero di campioni in un frame;

2. **indice relativo alla misurazione (N)**.

La formula per calcolare l'angolo di ogni misurazione è:

$$\text{Angolo} = \text{Angolo Start} + \frac{22.5}{M} \cdot N \quad (2.2)$$

- **byte (13+3N): separatore.**
Valore che separa i valori della distanza tra una misurazione e la successiva;
- **Byte (14+3N) e (15+3N): Distanza (MSB) e Distanza (LSB)**
Valore a 16 bit corrispondente alla distanza.
Per ottenere il valore effettivo della distanza, bisogna convertirlo da esadecimale in decimale e moltiplicarlo per 0.25 mm;
- **byte N-2 e N-1: CRC.**
Valore a 16 bit che rappresenta la somma cumulativa dal byte 0 di intestazione al byte $N - 3$, ovvero il valore precedente al primo byte relativo al CRC.

2.3.1 Esempio frame

Byte:	Valore:	Significato:
0	AA	Intestazione
1 e 2	00 9A	Lunghezza frame
3	01	Protocollo
4	61	Tipo frame
5	AD	Command word
6 e 7	00 92	Lunghezza payload
8	82	Velocità
9 e 10	00 87	Angolo di offset
11 e 12	69 78	Angolo di partenza
13	00	Separatore
14	00	Distanza (MSB)
15	00	Distanza (LSB)
16	46	Separatore
17	21	Distanza (MSB)
18	3A	Distanza (LSB)
...	...	...
N-5	5E	Separatore
N-4	5E	Distanza (MSB)
N-3	32	Distanza (LSB)
N-2	35	CRC
N-1	BC	CRC

Tabella 2.2: Struttura di un frame esempio

2.4 Voltage Level Converter

Il 4-Bit Bidirectional Voltage Level Converter, menzionato in 2.1.3, è stato impiegato per garantire la compatibilità tra il LiDAR e il microcontrollore utilizzato. Infatti il LiDAR ha una tensione in uscita sul PIN TX di 5V, mentre il microcontrollore (2.1.2) opera a una tensione di 3.3V.

Riferimenti: [1, 3, 4, 5, 2]

3 Acquisizione Dati

3.1 Funzione GetData()

Per l'acquisizione dei dati è stata implementata una funzione che consente di verificarne la corretta acquisizione.

Codice 3.1: Funzione getData().

```
1  int getData() {
2  while(Serial2.read() != (frame[0] = 170));
3  int i = 1;
4  uint16_t FRAME_LENGTH = 256;
5  while (i<FRAME_LENGTH+2) {
6      while(!Serial2.available());
7      frame[i] = Serial2.read();
8      if (i == 2) {
9          N=frame[1]<<8 | frame[2];
10     }
11     if (i == 3 && frame[3]!=1 ) return 1;
12     if (i == 4 && frame[4]!=97 ) return 1;
13     i++;
14 }
15 return 0;
16 }
```

All'avvio, la funzione attende che la lettura della seconda porta seriale individui il valore 170 (0xAA), corrispondente al byte 0 del messaggio relativo al frame (come descritto nella tabella 2.1), convertito in formato decimale dal microcontrollore. Una volta individuato il byte di intestazione, ha inizio un ciclo per il riempimento del frame, strutturato come segue:

1. vengono inizializzate due variabili: `i`, impostata a 1, e `FRAME_LENGTH`, inizializzata a 256. La prima rappresenta l'indice per il conteggio dei byte nel frame, mentre la seconda indica la dimensione del frame, che verrà successivamente aggiornata con il valore effettivo;
2. inizia il ciclo di riempimento del frame, che parte dal byte 1 (poiché il byte 0 è già stato inserito) e prosegue fino a due posizioni oltre la lunghezza del frame. Questa estensione è necessaria poiché il valore della lunghezza del frame include tutti i byte dall'intestazione fino a quello immediatamente precedente ai due byte finali del CRC;

3. prima di inserire i dati nel frame, la funzione attende la disponibilità di nuovi dati sulla porta seriale, garantendo che i byte letti siano effettivamente presenti prima di essere acquisiti;
4. al momento dell'inserimento del secondo byte, i primi due byte vengono combinati per determinare la lunghezza effettiva del frame, memorizzata nella variabile `FRAME_LENGTH`. Questi due byte contengono infatti l'informazione relativa alla dimensione del frame (come riportato nella tabella 2.1);
5. per assicurare la corretta sincronizzazione tra il sensore e il microcontrollore, vengono eseguite verifiche sui byte 3 e 4 del frame. Se questi valori non corrispondono alla struttura prevista (descritta nella tabella 2.1), la funzione restituisce il valore 1, segnalando un errore nell'acquisizione del frame;
6. nel caso in cui non vengano rilevati problemi di sincronizzazione, la funzione restituisce il valore 0, indicando che il riempimento del frame è avvenuto con successo e senza errori.

3.2 Estrazione Dati

Per fornire al software di visualizzazione i dati necessari, è stato indispensabile calcolare e gestire ulteriori parametri, tra cui la lunghezza del payload e l'angolo di partenza della scansione.

Codice 3.2: Gestione dati scansione.

```
1  uint16_t payloadLength = frame[6];
2  payloadLength <= 8;
3  payloadLength = payloadLength | frame[7];
4  uint16_t m = (payloadLength - 5) / 3;
5  uint16_t startAngle = frame[11];
6  startAngle <= 8;
7  startAngle = startAngle | frame[12];
8  float startAngleFloat = (float)startAngle / 100;
```

La lunghezza del payload è determinata dai byte 6 e 7 del frame, che vengono combinati nella variabile `payloadLength`. Quest'ultima viene successivamente utilizzata per calcolare il numero di misurazioni effettuate, applicando la formula 2.1.

L'angolo di partenza della scansione è contenuto nei byte 11 e 12 del frame e viene memorizzato nella variabile `startAngle`. Per garantire un'adeguata precisione nel calcolo, la variabile subisce un casting esplicito da intero a 16 bit senza segno a formato `float` e viene poi divisa per 100 (corrisponde a moltiplicare per 0.01°), ottenendo così il valore corretto in gradi.

3.3 Output del microcontrollore

Per trasmettere le informazioni al software di mappatura sviluppato in Python tramite comunicazione seriale, è stato necessario formattare in modo ordinato le misurazioni di angoli e distanze, adottando un metodo di rappresentazione ispirato alla struttura del messaggio relativo alla scansione di un frame da parte del sensore LiDAR.

Codice 3.3: Codice output seriale.

```
1  float angleVect[m];
2  float distanceVect[m];
3  for (int n=0; n<=m; n++){
4      angleVect[n] = startAngleFloat + 22.5f / m * n;
5      uint16_t distance = frame[14 + 3*n];
6      distance <= 8;
7      distance = distance | frame[15 + 3*n];
8      distanceVect[n]= distance * DISTANCE_RESOLUTION;
9  }
10 Serial.print("A");
11 for (int i =0; i<m; i++){
12     Serial.print(",");
13     Serial.print(angleVect[i]);
14     Serial.print(",");
15     Serial.print(distanceVect[i]);
16 }
```

Inizialmente, vengono dichiarati due vettori di dimensione `m` (numero di misurazioni effettuate). Successivamente, un ciclo `for` viene impiegato per il riempimento di entrambi i vettori:

- nel vettore `angleVect` vengono inseriti gli angoli relativi a ciascuna misurazione, calcolati secondo la formula 2.2;
- i dati relativi alle distanze, invece, vengono prima combinati all'interno di una variabile temporanea, quindi moltiplicati per la costante `DISTANCE_RESOLUTION` e successivamente memorizzati nel vettore `distanceVect`.

Una volta completato il riempimento dei vettori, i dati vengono trasmessi in uscita seguendo la struttura: A, angolo, distanza, angolo, distanza, ...

Riferimenti: [1, 2]

4 Visualizzazione Dati

Per valutare l'accuratezza dei dati acquisiti dal LiDAR, è stato realizzato un diagramma polare dei punti rilevati, confrontandolo col software fornito dall'azienda produttrice. A tale scopo, le due immagini: 4.1 e 4.2 mostrano rispettivamente la visualizzazione dei dati elaborati dal codice in Python e quella fornita dal produttore.

Il confronto tra queste due rappresentazioni evidenzia una notevole corrispondenza, dimostrando che i risultati ottenuti con l'implementazione fatta in laboratorio, sono estremamente simili a quelli ufficiali. Questa verifica sperimentale conferma l'affidabilità dell'approccio adottato e la corretta gestione del protocollo di comunicazione del sensore.

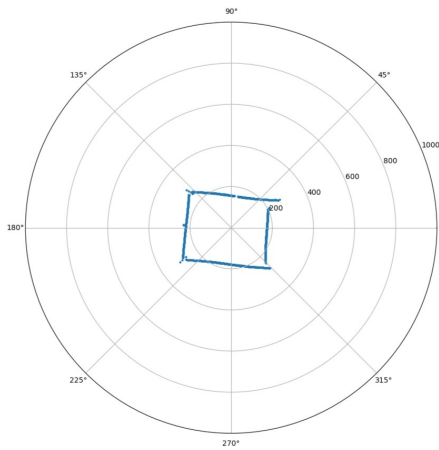


Figura 4.1: Diagramma polare scritto in Python

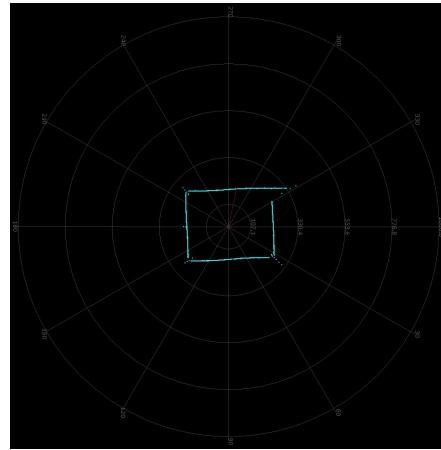


Figura 4.2: Diagramma polare offerto dall'azienda produttrice



Figura 4.3: Setup usato in laboratorio per la visualizzazione di un perimetro preciso e definito

4.1 Diagramma polare

Il diagramma polare è stato realizzato utilizzando il linguaggio Python all'interno dell'editor di testo Visual Studio Code. Questo strumento ha consentito una rappresentazione accurata dei dati acquisiti, visualizzandoli come punti caratterizzati da coordinate polari, ovvero angolo e distanza.

Il sensore è in grado di rilevare oggetti e ostacoli entro un raggio d'azione compreso tra un minimo di 12.8cm e un massimo di 8m, come riportato nel datasheet. La visualizzazione dei dati consente inoltre di ottenere informazioni dettagliate su ciascun punto semplicemente passando il cursore sopra di esso.

4.1.1 Rappresentazione polare

Seguendo la convenzione standard, il diagramma polare è stato rappresentato con l'asse degli 0° orientato verso destra e quello dei 180° verso sinistra. Diversamente, il software fornito dal produttore del sensore adotta una rappresentazione in cui gli 0° sono posizionati frontalmente.

4.2 Estrazione dati Python

```
1 def read_serial_data(self):
2     line = self.serial_port.readline().decode('utf-8').strip()
3     if line and line.startswith('A'):
4         try:
5             data = line[2:].split(',')
6             formatted_data = [(float(data[i]),
7                                float(data[i+1])) for i in range(0, len(data), 2)]
8             return formatted_data
9         except Exception as e:
10            print(f"Error parsing data: {e}")
11    return None
12
13 return None
```

La funzione `read_serial_data()` ha il compito di acquisire e interpretare i dati provenienti da una porta seriale. Essa legge una riga di dati dalla porta seriale, la decodifica in formato UTF-8 e rimuove eventuali spazi bianchi o caratteri di nuova riga presenti.

I dati in ingresso, contenenti informazioni sull'angolo e sulla distanza, vengono generati dal codice Arduino (descritto nel paragrafo 3.3) e successivamente inviati alla porta seriale, dalla quale vengono letti ed elaborati in Python.

Successivamente, il codice verifica che la riga acquisita non sia vuota e che inizi con il carattere A, il quale indica l'intestazione del messaggio. Se tale condizione è soddisfatta, il codice procede alla suddivisione dei dati, escludendo i primi due caratteri ("A,"), e separando i valori utilizzando la virgola come delimitatore.

I dati vengono poi convertiti in coppie di valori numerici in virgola mobile (`float`), effettuando un'iterazione sugli elementi ottenuti che vengono prelevati due alla volta e convertiti in una tupla della forma (`valore1, valore2`).

Se durante il parsing dei dati si verifica un'eccezione, viene stampato un messaggio di errore, e la funzione restituisce `None`. In tutti gli altri casi, i dati elaborati vengono restituiti sotto forma di una lista di tuple.

Se la riga acquisita non soddisfa le condizioni iniziali (ad esempio, se è vuota o non inizia con A), la funzione restituisce direttamente `None`, indicando che non sono presenti dati validi.

5 Debugging

Durante la fase di sviluppo del progetto, l'implementazione del sistema di acquisizione dati mediante il LiDAR Delta 2A ha presentato diverse sfide, sia a livello hardware che software. In particolare, sono emersi problemi relativi alla gestione del protocollo di comunicazione, alla compatibilità dei livelli di tensione e all'integrità dei dati ricevuti. Questa sezione descrive le principali problematiche riscontrate e le soluzioni adottate per garantire il corretto funzionamento del sistema. L'analisi e la risoluzione di questi problemi hanno rappresentato un passaggio fondamentale per ottimizzare l'affidabilità e la precisione delle misurazioni ottenute.

5.1 Problematiche riscontrate

Durante la realizzazione del progetto sono emerse diverse criticità, riassumibili nei seguenti punti:

1. incompatibilità con la piattaforma Arduino Uno R3;
2. disallineamento dei livelli di tensione tra il sensore e il microcontrollore;
3. gestione della comunicazione e raccolta dati tramite interfaccia seriale.

5.2 Analisi e risoluzione delle problematiche

5.2.1 Incompatibilità con Arduino Uno R3

Inizialmente, il progetto prevedeva l'impiego di una scheda Arduino Uno R3 come microcontrollore. Tuttavia, a causa della presenza di un'unica porta seriale hardware, utilizzata anche per la comunicazione con il visualizzatore, è stato necessario impiegare una seriale software per interfacciarsi con il sensore. Questa scelta ha introdotto una limitazione significativa: la velocità di trasmissione massima supportata dalla seriale software è pari a 115200 baud/s, mentre il LiDAR trasmette dati a una velocità di 230400 baud/s. Di conseguenza, i dati ricevuti risultavano corrotti e inutilizzabili (1).

Per ovviare a questo problema, si è optato per l'adozione del microcontrollore ESP32-WROOM-32 (2.1.2), il quale offre la possibilità di configurare una seconda porta seriale hardware su qualsiasi GPIO disponibile. Questa soluzione ha garantito una comunicazione stabile e affidabile con il sensore.

5.2.2 Disallineamento dei livelli di tensione

Il passaggio all'ESP32 ha portato alla seconda criticità evidenziata nel punto 2. Durante le prime fasi di test, è stato osservato che i pacchetti di dati ricevuti dal LiDAR risultavano errati. L'analisi del datasheet del sensore ha evidenziato che quest'ultimo opera a una tensione di 5V, mentre l'ESP32 a 3.3V. Tale disallineamento dei livelli logici ha compromesso la corretta trasmissione dei dati.

Per risolvere questa problematica, è stato introdotto un Voltage Level Converter (2.1.3), che ha consentito di adattare correttamente i livelli di tensione senza alterare l'integrità dei dati trasmessi. Grazie a questa soluzione, la comunicazione tra il LiDAR e ESP32 è stata stabilizzata, garantendo la corretta ricezione dei pacchetti.

5.2.3 Gestione della comunicazione e raccolta dati tramite porta seriale

In una fase iniziale del progetto, è stato riscontrato un problema di sincronizzazione dei dati con la porta seriale. Per risolverlo, è stato introdotto un delay di 10ms consentendo al buffer di svuotarsi e acquisire correttamente i nuovi dati in arrivo.

Successivamente, nell'ottimizzazione del programma, il delay è stato eliminato, sostituendolo con un nuovo tipo di comando, come illustrato nelle righe di codice 3.1.

L'ottimizzazione introdotta ha migliorato il throughput della trasmissione seriale, assicurando al contempo che tutti i pacchetti ricevuti avessero un CRC corretto (3).

6 Risultati e conclusioni

Il progetto ha permesso di studiare in dettaglio il funzionamento del LiDAR, la sua integrazione con un microcontrollore e l'ottimizzazione della comunicazione attraverso un codice dedicato in ambiente Arduino. L'implementazione del sistema con ESP32-WROOM-32 e l'utilizzo di un Voltage Level Converter hanno garantito un'acquisizione dati stabile e precisa, rispettando le specifiche del sensore.

I risultati ottenuti sono stati soddisfacenti, confermando le prestazioni attese in termini di risoluzione angolare e capacità di rilevamento. In particolare, il sistema ha raggiunto una risoluzione dell'angolo di 0.03° , con una distanza massima di 8m e una minima di 12.8cm, in linea con i valori riportati nel datasheet del sensore.

Questo lavoro ha dimostrato l'affidabilità della tecnologia LiDAR nel contesto analizzato e ha fornito una base solida per eventuali sviluppi futuri, come l'ottimizzazione degli algoritmi di elaborazione dati o l'integrazione con sistemi di controllo avanzati.

Bibliografia

- [1] Arduino. Arduino - Open Source Electronics Platform.
- [2] Birobotix. Delta-2a communication interface protocol.
- [3] Birobotix. Delta-2a data sheet.
- [4] Birobotix. Delta-2a-sdk.
- [5] Birobotix. Delta-2a user manual.